

CaveRunner Module 1

Module 1 — Project Setup & Core Game Loop

Goal

By the end of this module, CaveRunner should:

- open a desktop libGDX window,
- clear the screen every frame,
- load and draw a dirt texture and player texture,
- move the player using delta-time,
- wrap the player when it leaves the screen,
- cleanly dispose of rendering and asset resources.

This module establishes the foundation for every later module: project structure, the libGDX application lifecycle, asset loading, rendering, and delta-time.

1. Why libGDX, and how the project is shaped

CaveRunner uses **libGDX**, a Java game framework that provides a cross-platform game loop, rendering, input, audio, and file I/O. It does not provide the complete game logic for us; we build the world, player, physics, entities, and other systems ourselves.

The project is divided into modules so the same game logic can be reused across platforms.

```
CaveRunner/
|--- core/                # Platform-independent game code
|   |-- src/main/java/
|   |   |-- com/badlogic/CaveRunner/
|   |   |   |-- Main.java
|   |   |   |-- Config.java
|   |   |   |-- Assets.java
|   |
|   |-- lwjgl3/            # Desktop launcher
|   |   |-- src/main/java/
|   |   |   |-- com/badlogic/CaveRunner/lwjgl3/
|   |   |   |-- Lwjgl3Launcher.java
|   |
|   |-- android/          # Android module generated by GDX-Liftoff
|   |
|   |-- assets/           # Shared game assets
|   |   |-- dirt.png
|   |   |-- player.png
|   |
|   |-- build.gradle
|   |-- settings.gradle
|   |-- gradlew
|   |-- gradlew.bat
```

Important architecture rule

`core` contains the actual game logic and should remain platform-independent.

The desktop launcher in `lwjgl3` is responsible for creating the desktop application/window and starting `Main`.

Do not put desktop-specific windowing code inside `core`.

2. Project creation with GDX-Liftoff

For this project, the initial project was created using **GDX-Liftoff**.

Recommended choices:

Setting	Value
Template	Classic
Platforms	Desktop + Android + Core
Language	Java
Extensions	None initially
Project name	CaveRunner
Package	com.badlogic.CaveRunner

Why Classic?

The Classic template matches the simple libGDX architecture needed for this project.

We do not add Box2D or other extensions in Module 1 because the project is intentionally building its own gameplay systems step by step.

Package-name rule

GDX-Liftoff generated the package:

```
com.badlogic.CaveRunner
```

The original design document used:

```
com.badlogic.Terraria
```

For the actual project, **use one package name consistently**. We use:

```
com.badlogic.CaveRunner
```

Therefore:

```
package com.badlogic.CaveRunner;
```

and:

```
package com.badlogic.CaveRunner.lwjgll3;
```

Do not mix these package names.

3. Gradle configuration

GDX-Liftoff generates the Gradle configuration for the selected libGDX version and platform modules.

Important: Do not blindly replace the generated Gradle files with an older Gradle configuration from a tutorial or specification. First use the configuration generated by GDX-Liftoff.

The project specification uses the following conceptual structure:

```
root
|--- core
|---.lwjgll3
`--- android
```

The `core` module contains the game code.

The `lwjgl3` module depends on `core` and provides the desktop backend.

The Android module provides the Android platform integration.

Gradle commands

From the project root:

```
.\gradlew projects
```

Build the project:

```
.\gradlew build
```

Run the desktop game:

```
.\gradlew lwjgl3:run
```

Clean the build:

```
.\gradlew clean
```

4. `Config.java` — centralized game values

Create:

```
core/src/main/java/com/badlogic/CaveRunner/Config.java
```

```

package com.badlogic.CaveRunner;

/**
 * Every gameplay balance number lives here
 * so it is easy to tune.
 */
public final class Config {

    private Config() {
        // Prevent object creation.
    }

    // ---- General ----

    public static final String TITLE = "Cave Runner";

    // ---- World ----

    public static final int TILE = 16;
    public static final int WORLD_W = 300;
    public static final int WORLD_H = 120;
    public static final int SURFACE_BASE = 70;
    public static final int SPAWN_X = WORLD_W / 2;
    public static final int SPAWN_SAFE_RADIUS = 8;

    // ---- Player ----

    public static final float PLAYER_W = TILE * 0.75f;
    public static final float PLAYER_H = TILE * 1.75f;
    public static final float MOVE_SPEED = 120f;
    public static final float JUMP_VEL = 240f;
    public static final float GRAVITY = -600f;
    public static final float MAX_FALL = -400f;
    public static final int MAX_HEALTH = 100;
    public static final int START_LIVES = 3;

    // ---- Time ----

    public static final float DAY_LENGTH = 120f;
    public static final float AUTOSAVE_INTERVAL = 60f;

    // ---- Levels ----

    public static final int MAX_LEVEL = 5;
}

```

Why use a configuration class?

Instead of scattering values throughout the project:

```
playerX += 120f * delta;
```

we use:

```
playerX += Config.MOVE_SPEED * delta;
```

This makes balancing easier. Later modules can also use the same constants for physics and gameplay.

5. `Assets.java` — load textures once

Create:

```
core/src/main/java/com/badlogic/CaveRunner/Assets.java
```

```

package com.badlogic.CaveRunner;

import com.badlogic.gdx.assets.AssetManager;
import com.badlogic.gdx.graphics.Texture;
import com.badlogic.gdx.graphics.g2d.TextureRegion;
import com.badlogic.gdx.utils.Disposable;

import java.util.HashMap;
import java.util.Map;

/**
 * Loads textures once and provides them by name.
 */
public class Assets implements Disposable {

    private static final String[] NAMES = {
        "dirt",
        "player"
    };

    private final AssetManager manager = new AssetManager();

    private final Map<String, TextureRegion> regions = new HashMap<>();

    public Assets() {

        for (String name : NAMES) {
            manager.load(name + ".png", Texture.class);
        }

        manager.finishLoading();

        for (String name : NAMES) {

            Texture texture =
                manager.get(name + ".png", Texture.class);

            texture.setFilter(
                Texture.TextureFilter.Nearest,
                Texture.TextureFilter.Nearest
            );

            regions.put(
                name,
                new TextureRegion(texture)
            );
        }
    }

    public TextureRegion get(String name) {

        TextureRegion region = regions.get(name);

        if (region == null) {
            throw new IllegalArgumentException(
                "Missing texture: " + name
            );
        }

        return region;
    }

    @Override
    public void dispose() {
        manager.dispose();
    }
}

```

Why load assets once?

Creating/loading textures during every `render()` call would be inefficient.

Instead:

```

create()
  v
load assets once
  v
render()
  v
reuse assets

```

`TextureRegion` is used so later modules can use regions from texture atlases.

6. Required assets

Place these files in the shared assets directory:

```
assets/  
|--- dirt.png  
`--- player.png
```

For Module 1, simple placeholder pixel-art images or solid-color square PNGs are enough.

The names must match exactly:

```
dirt.png  
player.png
```

Do not use:

```
Dirt.png  
Player.png  
dirt.PNG  
player.PNG
```

The code requests the lowercase filenames.

7. `Main.java` — the core game loop

Create or replace:

```
core/src/main/java/com/badlogic/CaveRunner/Main.java
```

```
package com.badlogic.CaveRunner;  
  
import com.badlogic.gdx.ApplicationAdapter;  
import com.badlogic.gdx.Gdx;  
import com.badlogic.gdx.graphics.GL20;  
import com.badlogic.gdx.graphics.OrthographicCamera;  
import com.badlogic.gdx.graphics.g2d.SpriteBatch;  
  
/**  
 * Module 1:  
 * Proves the game loop, window, asset loading,  
 * delta-time movement, and cleanup.  
 */  
public class Main extends ApplicationAdapter {  
  
    private SpriteBatch batch;  
    private OrthographicCamera camera;  
    private Assets assets;  
  
    private float playerX = 100f;  
    private float elapsed = 0f;  
  
    private static final int VIEW_W =  
        32 * Config.TILE;  
  
    private static final int VIEW_H =  
        18 * Config.TILE;  
  
    @Override  
    public void create() {  
  
        batch = new SpriteBatch();  
  
        camera = new OrthographicCamera();  
  
        camera.setToOrtho(  

```

```

        false,
        VIEW_W,
        VIEW_H
    );

    assets = new Assets();
}

@Override
public void resize(int width, int height) {
    // Proper aspect-ratio handling comes in a later module.
}

@Override
public void render() {

    // -----
    // 1. UPDATE
    // -----

    float delta = Gdx.graphics.getDeltaTime();

    elapsed += delta;

    playerX += Config.MOVE_SPEED * delta;

    if (playerX > VIEW_W) {
        playerX = -Config.PLAYER_W;
    }

    // -----
    // 2. CLEAR
    // -----

    Gdx.gl.glClearColor(
        0.53f,
        0.75f,
        0.92f,
        1f
    );

    Gdx.gl.glClear(
        GL20.GL_COLOR_BUFFER_BIT
    );

    // -----
    // 3. DRAW
    // -----

    camera.update();

    batch.setProjectionMatrix(
        camera.combined
    );

    batch.begin();

    // Ground
    batch.draw(
        assets.get("dirt"),
        0,
        0,
        VIEW_W,
        Config.TILE
    );

    // Player
    batch.draw(
        assets.get("player"),
        playerX,
        Config.TILE,
        Config.PLAYER_W,
        Config.PLAYER_H
    );

    batch.end();
}

@Override
public void dispose() {

    batch.dispose();

    assets.dispose();
}
}

```

8. Understanding the libGDX lifecycle

The game lifecycle is:

```
create()
  v
render()
  v
render()
  v
render()
  v
...
  v
dispose()
```

`create()`

Called once when the application is initialized.

Used for:

- creating `SpriteBatch`,
- creating the camera,
- loading assets.

`render()`

Called repeatedly by libGDX.

The basic order is:

```
1. Update
2. Clear
3. Draw
```

`dispose()`

Called when the application closes.

Used to release resources such as:

```
batch.dispose();
assets.dispose();
```

We do not create our own:

```
while (true)
```

loop. libGDX manages the frame loop.

9. Why delta-time matters

This line gets the time since the previous frame:

```
float delta = Gdx.graphics.getDeltaTime();
```

Then:

```
playerX += Config.MOVE_SPEED * delta;
```

The speed is therefore expressed as:

```
pixels / second
```

rather than:

```
pixels / frame
```

For example, at approximately 60 FPS:

```
delta ~ 0.0167 seconds  
120 x 0.0167 ~ 2 pixels/frame
```

The exact number varies with frame time.

This same principle will later be used for:

- gravity,
- jumping,
- movement,
- animation,
- timers,
- day/night cycles.

10. `Lwjgl3Launcher.java`

The desktop launcher should be located at:

```
lwjgl3/src/main/java/com/badlogic/CaveRunner/lwjgl3/Lwjgl3Launcher.java
```

Use:

```
package com.badlogic.CaveRunner.lwjgl3;  
  
import com.badlogic.gdx.backends.lwjgl3.Lwjgl3Application;  
import com.badlogic.gdx.backends.lwjgl3.Lwjgl3ApplicationConfiguration;  
import com.badlogic.CaveRunner.Config;  
import com.badlogic.CaveRunner.Main;  
  
public class Lwjgl3Launcher {  
  
    public static void main(String[] args) {  
  
        Lwjgl3ApplicationConfiguration config =  
            new Lwjgl3ApplicationConfiguration();  
  
        config.setTitle(Config.TITLE);  
        config.setWindowedMode(960, 540);  
        config.useVsync(true);  
        config.setForegroundFPS(60);  
  
        new Lwjgl3Application(  
            new Main(),  
            config  
        );  
    }  
}
```

Important

If GDX-Liftoff generated an equivalent launcher containing additional configuration, keep those generated settings unless they conflict with the project.

The critical requirements are:

```
new Main()
```

being passed to:

```
new Lwjgl3Application(...)
```

and the package/imports matching the actual project package.

11. Run the game

From the project root:

```
.\gradlew lwjgl3:run
```

Expected result:

```

      |-----|
      |         |
      |  blue sky  |
      |         |
      |  player   |
      |#####|
      |         |
      |-----|

```

The player should continuously move from left to right and wrap around when it leaves the visible area.

12. Module 1 exercise

Change:

```
public static final float MOVE_SPEED = 120f;
```

to:

```
public static final float MOVE_SPEED = 400f;
```

Run:

```
.\gradlew lwjgl3:run
```

The player should move substantially faster.

Do **not** change this line:

```
playerX += Config.MOVE_SPEED * delta;
```

This demonstrates that gameplay speed is controlled through `Config.java`.

13. Verification checklist

Before starting Module 2, verify:

- ☒ GDX-Liftoff project created
- ☒ Classic template selected

- ☒ Desktop, Android, and Core modules created
- ☒ Java project created
- ☒ Package name is consistent
- ☒ `Config.java` created
- ☒ `Assets.java` created
- ☒ `Main.java` created
- ☒ Desktop launcher works
- ☒ `dirt.png` loads
- ☒ `player.png` loads
- ☒ Blue background appears
- ☒ Dirt strip appears
- ☒ Player appears
- ☒ Player moves
- ☒ Player wraps around
- ☒ Movement uses delta-time
- ☒ Resources are disposed
- ☒ `.\gradlew lwjgl3:run` works
- ☐ `MOVE_SPEED = 400f` exercise completed

14. Common problem: `ClassNotFoundException`

If Gradle reports something like:

```
Could not find or load main class
com.badlogic.CaveRunner.lwjgl3.Lwjgl3Launcher
```

check that the following three things agree.

File location

```
lwjgl3/src/main/java/com/badlogic/CaveRunner.lwjgl3/Lwjgl3Launcher.java
```

Package declaration

```
package com.badlogic.CaveRunner.lwjgl3;
```

Gradle main class

```
mainClassName = 'com.badlogic.CaveRunner.lwjgl3.Lwjgl3Launcher'
```

All three must describe the same class.

15. What Module 1 does NOT implement yet

Do not add these systems in Module 1:

- terrain generation,
- caves,
- ores,
- player physics,
- gravity,
- collision,
- enemies,

- combat,
- inventory,
- camera following,
- save system.

They belong to later modules.

Keeping Module 1 small makes it easier to identify whether the basic application architecture is working.

16. Module 2 preview

The next module replaces the hardcoded dirt strip with a generated tile world.

It will introduce:

```
Tiles.java
World.java
```

and a:

```
300 x 120
```

tile world with terrain generation, caves, and ore veins.

The progression is therefore:

```
Module 1
Window + Game Loop
  v
Module 2
World + Tiles
  v
Module 3
Camera + Rendering
  v
Module 4
Player Physics
  v
Later Modules
Gameplay Systems
```

Module 1 status

Complete when the game runs and the 400f movement exercise works.

Do not move to Module 2 until the basic loop is stable.